

CONTAINERS

1

WHY



Consistency: "Works on my machine"
→ same everywhere



Isolation: safe multi-tenancy
→ bounded resources & permissions



Efficiency: share OS kernel
→ fast start, low overhead

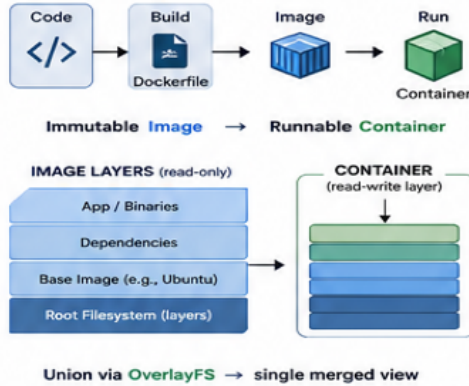


Portability: package once, run anywhere
→ standard images

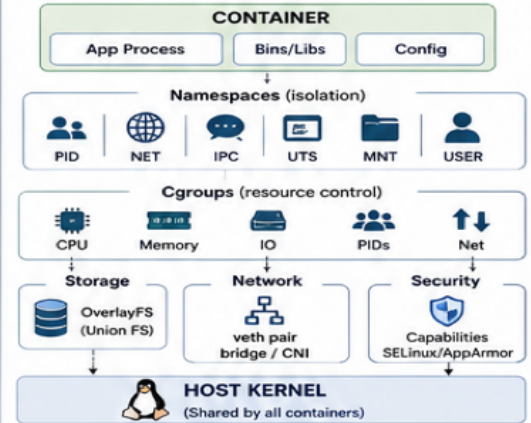


Scalability: build, ship, and scale
→ microservices friendly

2 CORE IDEA



3 INTERNALS



4 COMPLEXITY

Image Size	MBs-GBs Lower
Startup Time	100ms-seconds Fast
Runtime Overhead	~1-5% Low
Density (per host)	10s-1000s High
Isolation Strength	Process-level Medium
Operational Complexity	Medium with tooling

5 TRADE-OFFS

PROS	CONS
Lightweight	Share kernel → kernel bugs affect all
Fast start	Weaker isolation than VMs
High density	No true kernel customization
Portable images	State management is external
Efficient CI/CD	Noisy neighbor without limits
Ecosystem & tooling	Complex networking in multi-host
Immutable infra	Images must be rebuilt for changes

6 DISCUSSION

Containers vs VMs			Key Difference: Kernel Sharing
Aspect	Containers	VMs	
Kernel	Shared host kernel	Each VM has its own	
Isolation	Process-level	Hardware-level	
Startup	Seconds or less	Seconds to minutes	
Overhead	Low	High	
Density	High	Lower	
Use Cases	Microservices, CI/CD, Batch, APIs	Strong isolation, Different OS, Legacy apps	

Failure Modes

- Bad image / dependency
- Resource exhaustion
- Noisy neighbor
- Kernel vulnerability
- Misconfigured network/ACLs
- Orphaned volumes / state

Lifecycle

Build → Push → Pull → Run → Stop → Remove

- Ephemeral by design
- Replace, don't mutate
- State externalized (volumes, DBs, object storage)

7 RELATED

Alternatives & Complements

- VMs**
Stronger isolation, different OS
- microVMs (Firecracker)**
Lightweight VMs, stronger isolation
- Serverless**
No servers to manage, scale-to-zero
- Chroot / Jails**
Older isolation (weaker, limited)
- Sandboxes (gVisor, Kata)**
Stronger container isolation
- Kubernetes**
Orchestration, scaling, self-healing

Key Difference Summary

Containers share kernel → lightweight, fast, dense **VS** VMs have their own kernel → heavier, stronger isolation

8 PRODUCTION

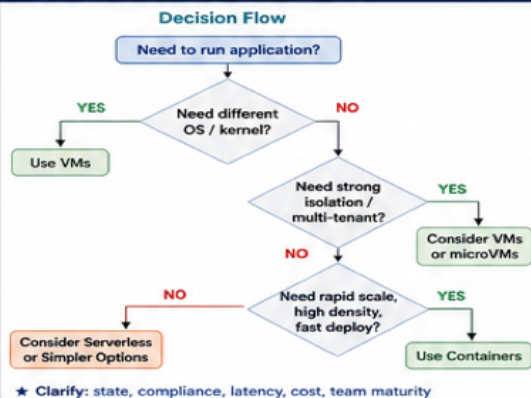
Where Containers Shine

- Microservices & APIs**
Independent deploy & scale
- CI/CD Pipelines**
Reproducible builds & consistent envs
- Batch & Workers**
Horizontal scale & resource limits
- Data Processing**
Spark/Flink jobs in containers
- Edge & IoT**
Lightweight, portable workloads
- Dev Environments**
Reproducible & on-demand

Common Tooling



9 INTERVIEW THINKING



★ Clarify: state, compliance, latency, cost, team maturity

10 SENIOR NOTES

- Design for failure: assume containers restart.
- Immutability: rebuild > SSH & patch.
- Least privilege: drop caps, read-only FS, non-root.
- Observe everything: logs, metrics, traces.
- Set SLOs & limits: CPU, memory, I/O, PIDs.
- Version & pin: base images, deps, digests.
- Keep it simple: small images, clear boundaries.

11 MENTAL HOOK

Containers package the app,
share the kernel, isolate the process,
control the resources.



Not a mini-VM. A secure, efficient **process envelope**.

BUILD ONCE ► **SHIP ANYWHERE** ► **RUN EVERYWHERE** ► **SCALE CONFIDENTLY**

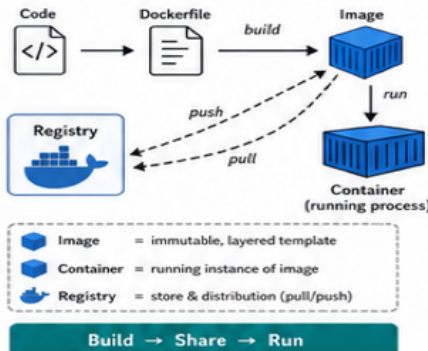
DOCKER

1 WHY

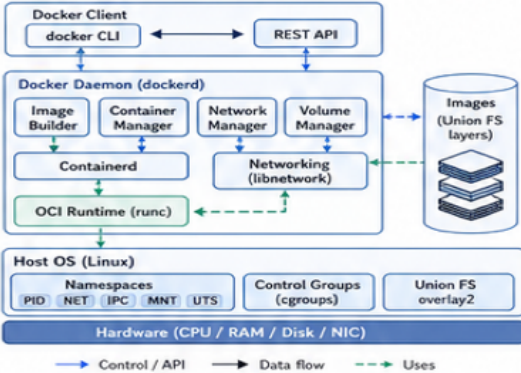
- Consistent everywhere**
Build once, run anywhere
- Isolation**
Process, filesystem, network boundaries
- Speed**
Fast start, lightweight sharing of OS kernel
- Portability**
Move apps across dev → test → stage → prod
- Density**
Run more apps per VM/host
- Ecosystem**
Huge toolchain, registry, marketplace

Rationale: Ship software faster with fewer "works on my machine" problems.

2 CORE IDEA

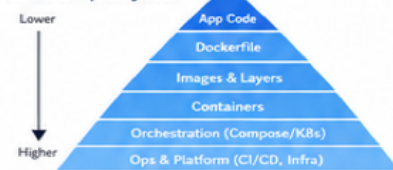


3 INTERNALS



4 COMPLEXITY

Where complexity lives



Typical cost drivers

- Image size & layers
- Number of containers
- Network topology & policies
- State & volumes
- Secrets & security controls
- Operational maturity

Principle: Keep it simple at each layer; add only what you need.

5 TRADE-OFFS

Choice	✓ Pros	⚠ Cons
Many small images	Fast pull, small blast radius	More layers, more mgmt
Fewer large images	Fewer layers, simpler mgmt	Slower pulls, bigger updates
Alpine (musl)	Tiny size	Compatibility, debugging
Debian/Ubuntu	Easier tooling, wider support	Larger size
Multi-stage builds	Small runtime images	More complex Dockerfile
Host network	Max performance	No isolation, port conflicts
Bridge network	Isolation, simple	NAT overhead, port mapping
Bind mounts	Easy sharing	Leaky abstraction, security risk
Volumes	Managed, portable	Extra setup
Root user	Simple	High risk
Non-root user	Safer	Permissions work

No silver bullet. Optimize for your constraints.

6 DISCUSSION

Key talking points

- Immutable infrastructure & reproducibility
- Isolation model: namespaces + cgroups
- Layered filesystem & copy-on-write (overlay2)
- Build cache for speed
- Supply chain & image provenance
- Observability & operability
- When to use Docker vs alternatives

Common failure modes

- ✗ "It works locally" → env drift / hidden deps
- ✗ Container exits → wrong CMD/ENTRYPOINT
- ✗ Can't connect → network / port mapping / firewall
- ✗ Data lost → ephemeral FS, no volume
- ✗ Image too big → bloat, wrong base, no pruning
- ✗ Permission denied → user/FS/volume perms
- ✗ Slow builds → no cache, invalidated layers
- ✗ Vulnerable image → outdated base, CVEs

7 RELATED

Alternatives / Complements

Kubernetes	Orchestration at scale
Podman	Daemonless, rootless option
containerd	Runtime used by Docker (and KBs)
BuildKit	Faster builds, better cache, secrets, mounts
CRI-O	KBs-native runtime
Nomad	Simple scheduler
Vagrant	VM-based environments

When to choose

Docker (Compose)	Local dev, small apps, simple stacks
Kubernetes	Large scale, HA, auto-scaling
Serverless (FaaS)	Event-driven, spiky, no server mgmt

8 PRODUCTION

Image hygiene

- ✓ Use official or trusted base images
- ✓ Pin versions (tags or digests @sha256)
- ✓ Minimize layers; combine RUN steps
- ✓ Multi-stage builds; copy only what you need
- ✓ Scan images (Trivy/Graype) in CI
- ✓ Sign images (cosign)

Runtime hygiene

- ✓ Run as non-root
- ✓ Read-only root FS
- ✓ Drop Linux capabilities
- ✓ No unnecessary services
- ✓ Health checks (HEALTHCHECK)
- ✓ Resource limits (CPU/Mem)

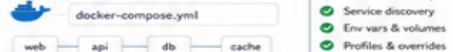
Data & state

- ✓ Volumes: Managed by Docker; persist across container restarts
- ✓ Bind mounts: Mount host paths; use carefully
- ✓ Backups: Don't forget your state & configs

Networking



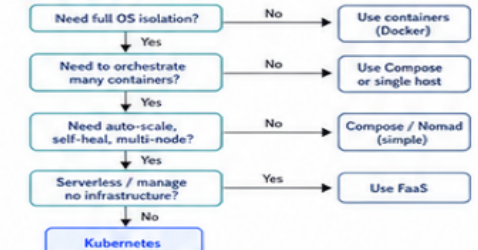
Compose (local multi-container)



- ✓ One command up
- ✓ Service discovery
- ✓ Env vars & volumes
- ✓ Profiles & overrides

9 INTERVIEW THINKING

Decision flow



What the interviewer listens for

- ✓ Clear trade-off reasoning
- ✓ Awareness of failures & mitigation
- ✓ Security & production mindset
- ✓ Experience with real-world ops

10 SENIOR NOTES

- Define & enforce standards: Base images, tooling, linting, tagging
- Design for operability: Logs, metrics, traces, alerts
- Plan for failure: Resiliency, rollback, idempotency
- Secure by default: Least privilege, scanning, patching
- Optimize continuously: Build times, image sizes, costs
- Mentor & document: Runbooks, ADRs, knowledge sharing

Heuristics

- ✓ Optimize the bottleneck
- ✓ Measure, don't guess
- ✓ Prefer boring & predictable
- ✓ Automate the repeatable
- ✓ Design for change

Anti-patterns

- ✗ Latest tag in prod
- ✗ Big mutable images
- ✗ Root user & full caps
- ✗ Secrets in images
- ✗ Unbounded logs
- ✗ No resource limits

11 MENTAL HOOK

Docker is a shipping container for code.



Layer it (Reuse work to go fast) | Share it (Push/Pull from registry) | Run it (Isolated, lightweight) | Manage it (Secure, observe, operate)

CHEAT SHEET

- Build image**
docker build -t myapp:1.0 .
- Run container**
docker run -d --name web -p 8080:80 myapp:1.0
- List**
docker ps -a
docker images
- Logs / Exec**
docker logs -f web
docker exec -it web sh
- Clean up**
docker system prune -a --volumes
- Push / Pull**
docker push myapp:1.0
docker pull myapp:1.0
- Volumes**
docker volume create data
docker run -v data:/data ...
- Networks**
docker network create mynet
docker run --network mynet ...
- Compose up**
docker compose up -d

Think in layers. Optimize for change. Secure by default. Operate with confidence.

Happy shipping!

KUBERNETES

1. WHY



Automate Ops
Deploy, scale, self-heal, roll back



Resilience
Survive failures across nodes & zones



Scale on Demand
Handle variable load efficiently



Portability
Run anywhere: cloud, on-prem, edge



Standard API
Extensible ecosystem, rich tooling

2. CORE IDEA

Declarative Desired State



Observe → Diff → Act → Reconcile → Repeat

Key Workloads & Objects

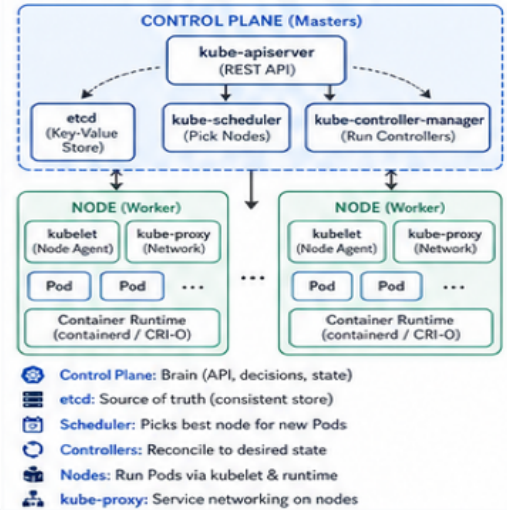


What Kubernetes Manages

- ✓ Scheduling
- ✓ Networking
- ✓ Storage
- ✓ Scaling
- ✓ Configuration
- ✓ Security
- ✓ Health
- ✓ Updates/Rollouts
- ✓ Failure Recovery

3. INTERNALS

Architecture Overview



4. COMPLEXITY

Conceptual	■■■■■	HIGH
Operational	■■■■■	HIGH
Learning Curve	■■■■■	HIGH
Moving Parts	■■■■■	HIGH
Debugging	■■■■■	HIGH
Payoff	■■■■■	VERY HIGH

Powerful but complex → Start simple, add features gradually.

5. TRADE-OFFS

Pros	Cons
✓ Self-healing	✗ Operational complexity
✓ Auto-scaling	✗ Steep learning curve
✓ High availability	✗ Resource overhead
✓ Portability	✗ More moving parts
✓ Ecosystem	✗ YAML sprawl
✓ Immutable infra	✗ Debugging can be harder

Great defaults for 80%, know when it's overkill.

6. DISCUSSION

K8s vs Alternatives

Feature	Kubernetes	Docker Compose	Serverless (e.g., Lambda)
Orchestration & Scheduling	★★★★★	★	★★
Auto-scaling	★★★★★	★	★★★★★
Self-healing	★★★★★	★	★★★★★
Portability	★★★★★	★★★	★★★
Operational Overhead	High	Low	Low
Best For	Long-running services at scale	Local dev / simple apps	Event-driven / burst workloads

Not either/or → Many systems use a mix.

7. RELATED

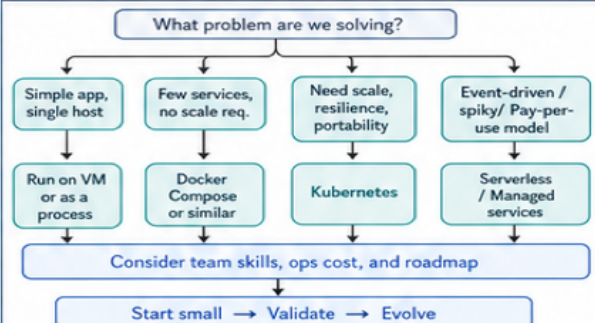
Helm	→ Package & deploy K8s apps
kustomize	→ Template & overlay configs
Ingress	→ HTTP(S) routing to Services
Service Mesh (Istio)	→ Traffic mgmt, security, obs
Prometheus	→ Metrics collection
Grafana	→ Dashboards & visualization
EFK/ELK	→ Logging pipeline
Argo CD / Flux	→ GitOps for delivery
CSI	→ Storage integration
CNI (Calico, Cilium)	→ Networking plugins

8. PRODUCTION

- Design for SLOs**
 - Define SLOs & alerts
 - Capacity & performance testing
- High Availability**
 - Multi-node control plane
 - Multi-AZ worker nodes
- Resilience**
 - PodDisruptionBudgets
 - Anti-affinity, topology spreads
- Security**
 - RBAC, NetworkPolicies
 - Secrets management (Vault, Sealed)
- Observability**
 - Metrics, logs, traces
 - Alert on errors & saturation
- Upgrades & Ops**
 - Rolling upgrades
 - Backups of etcd, DR plan

Automate, observe, iterate.

9. INTERVIEW THINKING

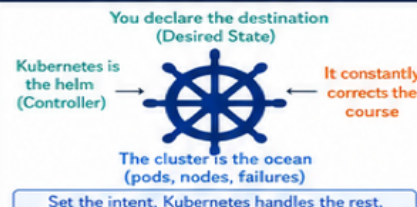


10. SENIOR NOTES

- ★ **Great Defaults** Use built-ins before custom solutions.
- ★ **Design for Change** Embrace immutable & declarative.
- ★ **Observe Everything** You can't fix what you can't see.
- ★ **Minimize Blast Radius** Namespaces, limits, quotas.
- ★ **Automate Operations** GitOps, CI/CD, policy as code.
- ★ **Cost Awareness** Right-size, autoscale, bin-pack.
- ★ **People & Process** Docs, runbooks, on-call readiness.

Technical + Operational excellence = Reliable platform

11. MENTAL HOOK



QUICK REFERENCE (KEY CONCEPTS)

- Pod** Smallest deployable unit; 1+ containers sharing network & storage.
- Deployment** Manages ReplicaSets; declarative updates & rollbacks.
- ReplicaSet** Ensures desired number of Pods are running.
- Service** Stable networking & load balancing for Pods.
- ConfigMap** Non-sensitive configuration as key/value.
- Secret** Sensitive data (passwords, tokens, certs).
- Probe** Health checks: Liveness (is alive), Readiness (can serve).
- HPA** Autoscale based on metrics (CPU/custom).
- Node** Worker machine that runs Pods.
- Reconciliation** Control loop that fixes actual state to match desired state.

DECLARE THE INTENT. AUTOMATE THE REST. KEEP IT SIMPLE. MAKE IT RELIABLE.



SERVICE MESH

1

WHY



Service-to-service calls are hard
Failures, latency, retries, security, observability, versioning



Centralize cross-cutting concerns
Consistent policy & telemetry across all services



Decouple apps from plumbing
App teams ship features; mesh handles traffic & security



Evolve safely
Traffic shifting, canary, A/B, gradual rollouts



Zero trust by default
mTLS everywhere, workload identity

2

CORE IDEA

DATA PLANE

Handles traffic

- ✓ L7 proxy (sidecar or ambient) in each workload
- ✓ Interception, routing, retries, mTLS, metrics
- ✓ Fast path, local decisions

CONTROL PLANE

Configures & observes

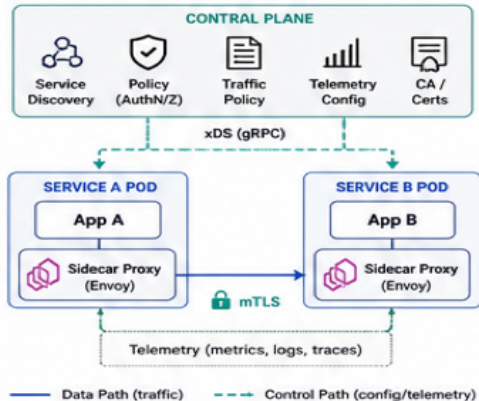
- ✓ Service discovery
- ✓ Traffic policies
- ✓ Security (mTLS, authz, certs)
- ✓ Telemetry config & collection
- ✓ xDS API push (or pull)

OUTCOME

Secure, observable, reliable service-to-service communication with uniform policy

3

INTERNALS



4

COMPLEXITY

ADDED COMPONENTS

- Sidecar proxies (or ambient dataplane)
- Control plane (multi-component)
- CRDs / config / RBAC
- Cert management & rotation
- Telemetry pipeline (metrics, logs, traces)



OPERATIONAL LOAD

Deployment: High
Day-2 Ops: High
Debugging: Medium
Learning curve: High

➔ Powerful, but not "set and forget"

5

TRADE-OFFS



Reliability, observability, security

Latency overhead



Centralized policy, consistency

Operational complexity



Fine-grained traffic control

Config sprawl, misconfig risk



mTLS, zero trust, strong identity

CPU/Memory, handshake cost



Rich telemetry, deep insights

Cost (infra, storage, egress)

★ Use where network complexity justifies the cost.

6

DISCUSSION

TALK TRACK

- ✓ Data plane (proxies) handle traffic locally.
- ✓ Control plane pushes config via xDS.
- ✓ All service-to-service traffic mTLS by default.
- ✓ Policies: routing, retries, timeouts, circuit breaking, fault injection, rate limiting.
- ✓ Observability: metrics, logs, traces with context.

COMMON FOLLOW-UPS

- Sidecar vs Ambient?
- How does discovery work?
- How are configs delivered?
- What's the performance impact?
- How do you debug a failed request?
- How do you roll out safely?



7

RELATED

Approach	Pros	Cons	Best When
Service Mesh (Sidecar)	Rich L7, policy, observability, mTLS	Higher overhead, operational complexity	Complex traffic needs, multi-team orgs
Service Mesh (Ambient)	Lower overhead, no sidecars	Feature gaps (L7), newer ecosystem	Large scale, latency/efficiency sensitive
API Gateway	North-South traffic, auth, rate limit	Not for service-to-service	External clients
Library/SDK	Minimal infra, lightweight	App changes, inconsistent implementation	Simple needs, few services
No Mesh	Lowest complexity, no overhead	App-owned plumbing, inconsistent	Small systems, stable teams

8

PRODUCTION

KEY CAPABILITIES



mTLS & Identity



Traffic Management



Resilience (Retries/CB)



Telemetry & Tracing



AuthZ & RBAC

OPERATIONAL CHECKLIST

- ✓ Start with observability → then policies → then advanced.
- ✓ Define SLOs and error budgets.
- ✓ Protect control plane (HA, backups, upgrades).
- ✓ Capacity plan proxies (CPU/Memory) & nodes.
- ✓ Alert on 5xx, latency, saturation, mTLS failures.
- ✓ Have rollback plan for bad configs.
- ✓ Continuously validate with canaries & chaos tests.

COMMON FAILURE MODES



Control plane down



mTLS / cert issues



Misrouted traffic



Proxy overload

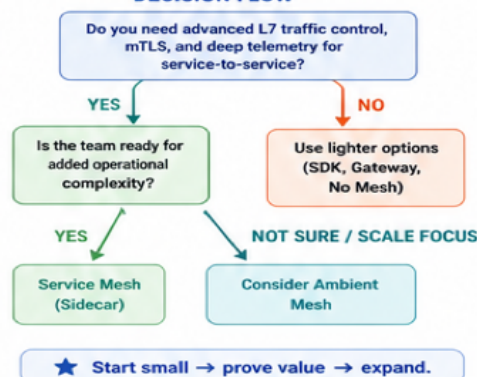


Bad config rollout

9

INTERVIEW THINKING

DECISION FLOW



10

SENIOR NOTES



Align with org model: platform team owns mesh; service teams own their code.



Govern defaults, allow safe overrides.



Cost visibility: proxies, telemetry, egress.



Invest in skills, runbooks, and gold-path docs.



Continuously evolve policies with real data.

★ A mesh is a platform—treat it like one.

11

MENTAL HOOK



GUARDS (Security)



TRAFFIC COP (Control)



BLACK BOX (Observability)



SERVICE MESH

The mesh is the invisible network your services rely on.
Secure. Smart. Observable.



DATA PLANE = PROXIES (FAST PATH)

CONTROL PLANE = BRAIN (POLICY & DISCOVERY)



mTLS BY DEFAULT



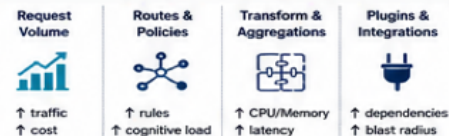
OBSERVE → DECIDE → ACT

API GATEWAY

1 WHY

- Single entry point for all clients simplifies clients & security
- Cross-cutting concerns in one place security, rate limiting, observability
- Route to right service hide internal topology
- Evolve backend independently versioning, transformation
- Improve reliability resilience, caching, traffic control
- Policy enforcement & governance consistency at scale

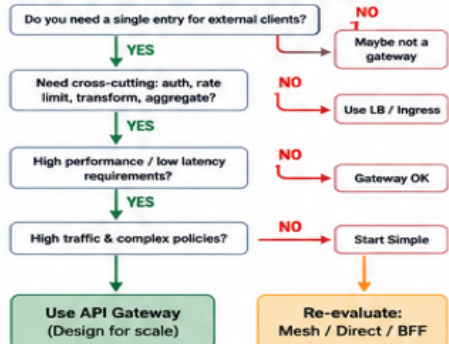
4 COMPLEXITY



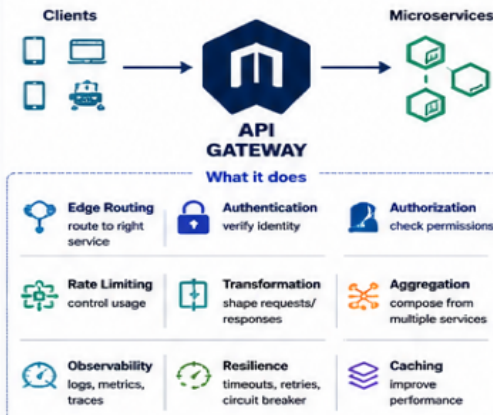
7 RELATED

- Service Mesh** (e.g., Istio, Linkerd) L7 traffic mgmt inside cluster; gateway is generally north-south
- Ingress Controller** (NGINX, Traefik) L7 HTTP routing into Kubernetes; gateway adds policies, transforms, aggregation
- CDN** (CloudFront, Fastly) Edge caching, static content, global accel; not for authz/transform/aggregation
- Load Balancer** (ALB/NLB) L4/L7 routing, TLS termination; limited policies & transforms
- BFF** (Backend for Frontend) Tailored APIs per client; complements gateway, not a replacement
- Service-to-Service** (Direct) Internal calls bypass gateway for lower latency & cost

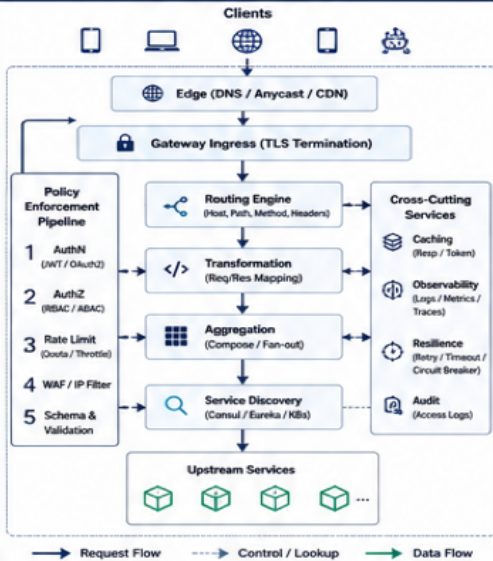
10 INTERVIEW THINKING



2 CORE IDEA



3 INTERNALS



8 PRODUCTION



5 TRADE-OFFS

BENEFITS	COSTS / RISKS
Simplifies clients	Adds hop (latency)
Centralized security & governance	Centralized failure (bottleneck, SPOF)
Policy consistency at scale	Increased cost (Infra & ops)
Faster backend evolution	Operational complexity
Observability in one place	Can hide issues (magic box)
Traffic control & protection	Misconfig = wide impact

6 DISCUSSION

Feature	API Gateway	Service Mesh	Ingress/LB
Scope	Edge (N-S) + some E-W	E-W (within cluster)	Edge (N-S)
Layer	L7 (HTTP/gRPC)	L7 (mTLS)	L4/L7
Cross-cutting Policies	Rich (Auth, Rate Limit, Transform, Aggregate)	Rich (mTLS, Retries, Traffic Mgmt)	Limited
Transforms & Aggregation	Yes	Limited	No
Client-facing APIs	Yes	No	Yes (Limited)
Observability	Centralized	Distributed	Limited
Typical Tools	Kong, AWS APIGW, Apigee, NGINX+	Istio, Linkerd	NGINX Ingress, ALB, Traefik
When to Use	External APIs, policies, transform, rate limiting	Service-to-service communication, mTLS	Simple routing into cluster

9 SENIOR NOTES

- Keep it thin: enforce policies, route, observe. Avoid business logic in gateway.
- Prefer fail-closed for authN/Z; be explicit about fail-open for availability (and why).
- Design for multi-tenancy: per-tenant limits, isolation, noisy neighbor protection.
- Version your APIs & policies; deprecate gracefully.
- Measure everything: optimize P99, not avg.
- Cache smart: tokens, configs, idempotent responses.
- Minimize transformations; a contract is a budget.
- Test like prod: load, chaos, dependency failures.
- Gateway is a product: UX (errors), docs, SLAs.

11 MENTAL HOOK

API GATEWAY = TRAFFIC COP AT THE EDGE



Right policies. Fast path. Clear visibility.
Secure, reliable APIs at scale.

TERRAFORM

DECLARATIVE INFRASTRUCTURE AS CODE • PROVISION • CHANGE • VERSION • REPEAT

1 WHY



Consistency
Same environment,
every time



Safety
Review, plan, and
predict changes



Speed
Automate repeatable
infrastructure

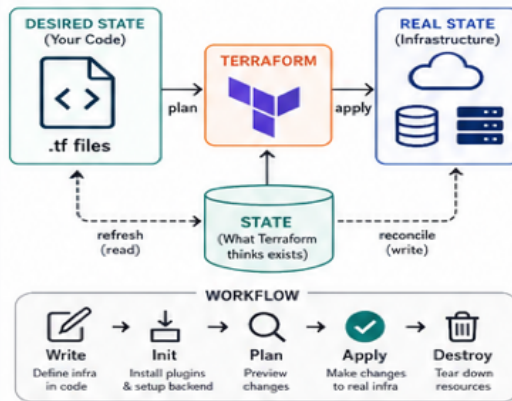


Collaboration
Versioned, reviewed,
team workflow

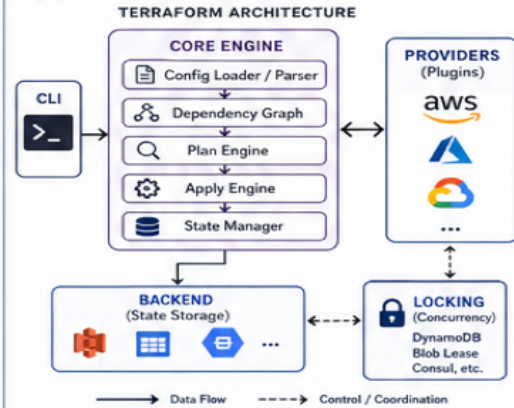


Cost Control
Right-size, track, eliminate
drift & waste

2 CORE IDEA



3 INTERNALS



4 COMPLEXITY

WHAT DRIVES COMPLEXITY



APPROX COMPLEXITY (EFFORT TO MASTER)

BASIC (1-2 wks)	INTERMEDIATE (1-3 mo)	ADVANCED (3-6 mo+)
- Resources - Plan/Apply - State basics	- Modules - Workspaces - Remote state - Outputs	- Providers - Policy as Code - Automation - Large scale

5 TRADE-OFFS

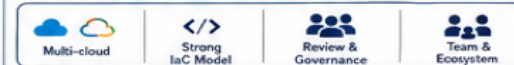
TRADE-OFF	GOOD	TRADE-OFF
Speed	Fast to provision & iterate	Plan/Apply time Grows with scale
Safety	Plan, review, versioned	State is critical If lost/corrupt = risk
Abstraction	Modules reduce repetition	Too much indirection Harder to debug
Cost Control	Visibility & drift remediation	State reads cost APIs & refresh
State Locking	Prevents conflicts	Lock contention Can block progress
Single Tooling	One workflow end-to-end	Provider limits Not every feature immediately

6 DISCUSSION

TERRAFORM vs ALTERNATIVES

TOOL	TYPE	STRENGTHS	LIMITATIONS
Terraform	Declarative (IaC)	Multi-cloud, rich ecosystem, graph, plan & state	State mgmt Complexity
AWS CloudFormation	Declarative (IaC)	Tight AWS integration	AWS-only, Less flexible
Pulumi	Imperative + Declarative	Real languages, strong typing	Smaller ecosystem
Ansible	Imperative (Config Mgmt)	Simple, agentless, great for ops	Not true desired state, drift
AWS Cloud Development Kits (CDK)	Imperative (IaC)	Use real code, reuse libraries	Lock-in, Synthesis step

WHEN TO CHOOSE TERRAFORM



7 RELATED



Key Concepts

- Declarative IaC
- Providers / Resources / Data Sources
- State
- Plan / Apply / Destroy
- Dependency Graph
- Modules
- Workspaces
- Variables / Outputs



Ecosystem

- Registry (modules & providers)
- Sentinel / OPA (policy)
- TFC / TFE (run, state, policies)



Patterns

- Blue/Green Infra
- Environment per workspace
- Remote state sharing (data sources)
- GitOps for infra

8 PRODUCTION

PRODUCTION CHECKLIST

- ✓ Remote State: S3 + DynamoDB / Azure / GCS
- ✓ State Locking: Always enabled
- ✓ Access Control: Least privilege IAM / RBAC
- ✓ Secrets Management: No secrets in state; use Vault/SSM/Key Vault/Secrets Manager
- ✓ CI/CD Integration: Plan on PR, Apply via pipeline
- ✓ Policy as Code: Sentinel / OPA / tfsec / Checkov
- ✓ Drift Detection: Scheduled plan / drift alerts
- ✓ Backups: Versioning + state backups
- ✓ Observability: Logs, metrics, cost tags

COMMON FAILURE MODES



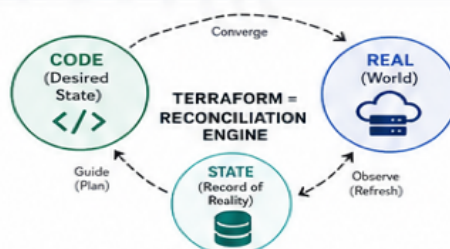
9 INTERVIEW THINKING



10 SENIOR NOTES

- ★ Design for operability, not just delivery.
- ★ State is your source of truth—protect it like prod data.
- ★ Prefer small, composable modules with clear contracts.
- ★ Expose outputs deliberately; avoid tight coupling.
- ★ Use workspaces sparingly; prefer separate state files for isolation.
- ★ Automate guardrails: policy, checks, approvals.
- ★ Plan cost, quotas, and limits up front.
- ★ Think in dependency graphs, not linear steps.
- ★ Keep secrets out of state; use references, not values.
- ★ Document conventions; make the easy path the right path.

11 MENTAL HOOK



REMEMBER

- ✓ Declare → Plan → Apply
- ✓ State → Lock → Safety
- ✓ Graph → Order → Change
- ✓ Drift → Detect → Fix
- ✓ Modules → Reuse → Scale

**You describe it.
Terraform makes it so.**

CODE → PLAN → APPLY → VERIFY → OPERATE → IMPROVE

INFRASTRUCTURE AS CODE, DONE RIGHT.

OBSERVABILITY

1 WHY



Understand system behavior
Know what's happening



Detect issues early
Fast feedback, less blast radius



Debug & resolve faster
From hours → minutes



Build trust & reliability
Data-driven decisions



Optimize performance & cost
Right signals, right spend



You can't improve what you can't observe.

2 CORE IDEA

Metrics



Logs



Traces



What
Aggregated
facts

Why
Events with
context

How
Requests
end-to-end

Infer internal state from external outputs
→ Make the unknowns known.

GOLDEN SIGNALS (for Services)



Latency
Time to serve



Traffic
Demand rate



Errors
Failures rate



Saturation
Utilization of resources

SLI / SLO

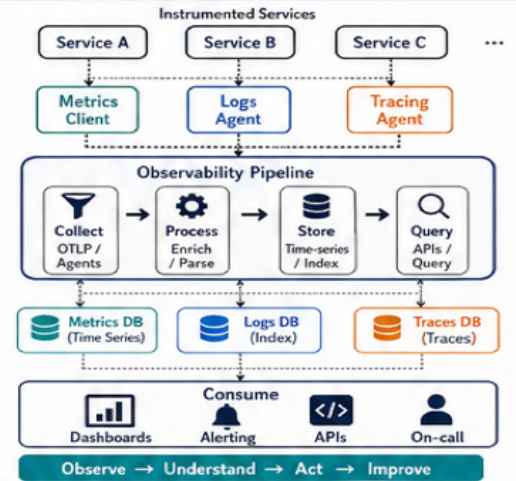
SLI
Goodness of service

SLO
Target for SLI
(e.g., 99.9%)



Error Budget
How much we can burn

3 INTERNALS



4 COMPLEXITY



High cardinality
→ Explodes cost & noise



High volume
→ Storage, ingest, retention



Distributed systems
→ Many hops, many failures



Context loss
→ Hard to correlate



Sampling bias
→ Can hide rare issues



Alert fatigue
→ Too noisy, gets ignored



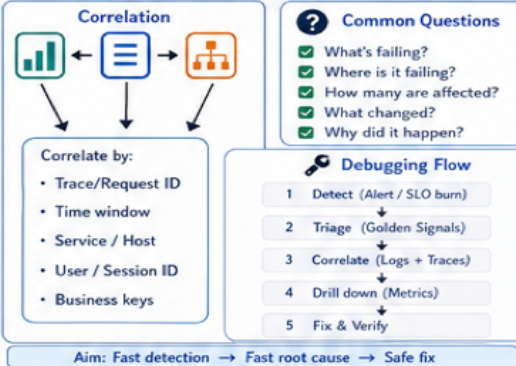
Observability is a system, not just tooling.

5 TRADE-OFFS

Decision	More of This	Leads To
Granularity	More detail (labels, fields)	Better insight Higher cost
Cardinality	More dimensions	Richer queries Higher cost
Sampling	Higher sample rate	Better fidelity Higher cost
Retention	Longer retention	Better forensics Higher cost
Real-time	Lower latency alerts	Faster detection More noise
Automation	More alerts / rules	Earlier signals Risk of noise

⚖️ Balance: Insight vs Cost vs Noise

6 DISCUSSION



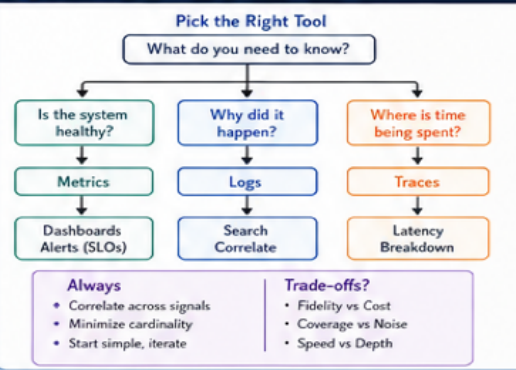
7 RELATED

Approach	What It Gives	Best For
Metrics	Aggregated trends, rates, ratios	Monitoring, alerting
Logs	Detailed events with context	Debugging, audit, search
Traces	Request flow, latency breakdown	Distributed debugging
Profiling	CPU/Memory usage	Performance deep dives
Synthetics	Outside-in monitoring	User journey, uptime
RUM	Real user metrics (web/mobile)	UX, frontline performance

8 PRODUCTION

Real-World Examples	Key Practices
E-commerce High latency checkout → Traces show slow payment gateway	✓ Define SLIs & SLOs per service
Streaming Error spike on video start → Metrics + Logs show CDN 5xx	✓ Dashboards for Golden Signals
Payments SLO burn on success rate → Correlate with third-party errors	✓ Alert on SLO burn (not just thresholds)
Microservices Increased p99 latency → Trace reveals N+1 DB calls	✓ Use trace IDs everywhere
Job failures at night → Logs show bad input schema	✓ Keep labels low-cardinality
	✓ Sample smartly
	✓ Review & remove unused data
	✓ Postmortems & learn continuously

9 INTERVIEW THINKING



10 SENIOR NOTES

- ★ Design for operability, not just functionality.
- ★ Treat observability as a product.
- ★ Define SLIs from user perspective.
- ★ Alert on outcomes (SLO burn), not symptoms.
- ★ Budget your telemetry (cost + cardinality).
- ★ Build shared libraries & standards.
- ★ Automate, but keep humans in the loop.
- ★ Continuously prune & optimize data.
- ★ Practice chaos & game days.
- ★ Make learning from incidents habitual.



Principle: Observe the right things, at the right cost, for the right people.

11 MENTAL HOOK

OBSERVABILITY = X-RAY + VITALS + BLACK BOX



X-RAY (Traces)
See through the system (flow)



VITALS (Metrics)
Is it healthy? (numbers)



BLACK BOX (Logs)
What happened? (events)

All three. Correlated. Contextual. Actionable.
That's how you turn unknowns into answers.

Failure Modes

- Too many alerts
→ Alert fatigue
- Missing context
→ Hard to debug
- High cardinality
→ Cost explosion
- Sampling blind spots
→ Miss critical issues
- Long MTTR
→ Customer impact

Watch For

- Spikes in error rate
- SLO burn rate
- Latency p95/p99
- Resource saturation
- Traffic anomalies
- Dependency failures
- Deployment changes

Measure → Understand → Correlate → Act → Improve

Better signals. Better decisions. Better systems.

LOGGING

1 WHY

 Observability What's happening?	→ See, measure, understand
 Debugging Why did it fail?	→ Reproduce, root cause
 Incident Response What happened?	→ Timeline, impact, blast radius
 Audit & Compliance Who did what?	→ Traceability, evidence
 Business Insights How is it performing?	→ Trends, usage, SLOs

Logs = Immutable timeline of system truth

2 CORE IDEA

Emit structured events with context.

```
{
  "ts": "2025-05-11T12:34:56.789Z",
  "level": "ERROR",
  "msg": "Payment failed",
  "logger": "checkout.service",
  "service": "checkout",
  "env": "prod",
  "version": "1.3.7",
  "trace_id": "4bf92f3577b34da6",
  "span_id": "00f067aa0ba902b7",
  "correlation_id": "order_abc123",
  "user_id": "u_42",
  "order_id": "o_987",
  "duration_ms": 312,
  "meta": {"retry": 2, "payment": "stripe"}
```

 ts	When it happened (ISO-8601 UTC)
 level	SEVERITY: TRACE < DEBUG < INFO < WARN < ERROR
 msg	Human-readable summary
 logger	Source (module/class/component)
 trace_id / span_id	Distributed tracing correlation
 correlation_id	Business request / workflow ID
 context	Who, what, where, custom fields

3 INTERNALS

End-to-end logging pipeline



- Key Internals**
- ✓ Append-only writes → immutable logs
 - ✓ Sharding & replication → durability & scale
 - ✓ Compression (gzip/zstd) → reduce storage & cost
 - ✓ Partition by time (e.g., daily/hourly) → prune fast

4 COMPLEXITY

 Emit Logs	LOW
 Structured Events	LOW
 Context / Correlation IDs	MEDIUM
 Centralized Collection	MEDIUM
 Indexing & Search	MEDIUM
 Retention / Lifecycle	MEDIUM
 Redaction / PII Handling	HIGH
 Sampling / Rate Limiting	MEDIUM
 Multi-tenant Isolation	HIGH
 High Volume (Scale)	HIGH
 Cost Optimization	HIGH
 Alerting / Anomaly Detection	MEDIUM

LOW = easy MEDIUM = moderate HIGH = hard

5 TRADE-OFFS

 More Logs (Detail)	↔	Higher Cost (Storage/Compute)	
 Rich Context (Easier Debug)	↔	Latency & Payload Size	
 Fine Granularity (Every Event)	↔	Noise & Signal-to-Noise	
 Long Retention (Coverage)	↔	Cost & Storage Growth	
 Real-time Search (Fast Insights)	↔	Index Cost & Write Amplification	
 PII in Logs (Useful)	↔	Privacy & Compliance Risk	
 No Sampling (Completeness)	↔	High Volume & Cost	
 Self-hosted (Control)	↔	Ops Overhead (Manage Infra)	
 Managed Service (Simplicity)	↔	Vendor Lock-in & Cost	

6 DISCUSSION

- When to Log**
- ✓ State changes, important business events
 - ✓ Errors, retries, timeouts
 - ✓ External calls (req/resp, latency)
 - ✓ Security & audit events
 - ✓ Anomalies & SLO violations

- What NOT to Log**
- ✗ Secrets, passwords, tokens, PII
 - ✗ High-cardinality spam (e.g., per-item debug in loops)
 - ✗ Large payloads / blobs (store reference instead)
 - ✗ Expected noise (health checks, 200 OK spam)

Lower Severity			Higher Severity		
TRACE	DEBUG	INFO	WARN	ERROR	FATAL
Most detailed	Developer detail	General ops	Something unexpected	Failures impacting requests	Service cannot continue

Rule of Thumb: INFO in prod, DEBUG in non-prod, ERROR always

7 RELATED

 Metrics	→ Aggregated numbers (what & how much)
 Traces	→ Request flow & latency (why & where)
 Events	→ Business facts across systems
 Alerts	→ Notify on conditions (what to act on)
 Dashboards	→ Visualize logs/metrics at a glance
 APM	→ Correlate logs, metrics, traces together

Use together: Logs explain, Metrics quantify, Traces connect the dots.

8 PRODUCTION

Best Practices

- ✓ Use structured logs (JSON)
- ✓ Include context: trace_id, correlation_id, user_id
- ✓ Consistent schema & field names
- ✓ Set correct levels: avoid over-logging
- ✓ Redact PII/Secrets at source
- ✓ Sample high-volume logs
- ✓ Define retention & lifecycle
- ✓ Monitor logging pipeline (ingestion lag, errors)
- ✓ Alert on errors, spikes, silent failures

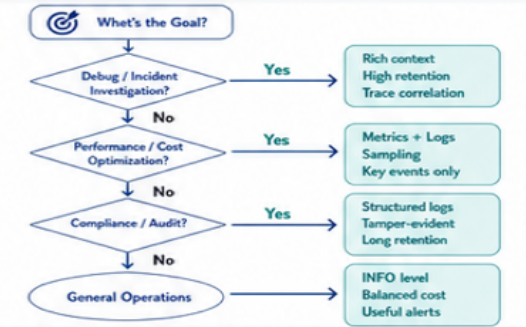
Sampling Guide	
ERROR	100%
WARN	100%
INFO	10–50%
DEBUG	1–5%
TRACE	0.1–1%

Retention Strategy (Example)



✓ Comply with legal & data retention policies.

9 INTERVIEW THINKING



10 SENIOR NOTES

- ★ Think in terms of cost per insight.
- ★ Design for queryability: stable schema, good defaults.
- ★ Treat logging pipeline as a critical system: monitor it.
- ★ Protect data: least-privilege access, encryption at rest/in transit.
- ★ Plan for scale: volume grows 10–100x over time.
- ★ Favor logs that answer questions, not just print everything.
- ★ Evolve schemas carefully; version when needed.
- ★ Document logging standards & examples for teams.
- ★ Continuously review top queries, costs, and retention.
- ★ Be ready to justify every log in prod.



11 MENTAL HOOK

Logs are a time machine.



Capture once. Search forever. Act faster.

12 FAILURE MODES

 Logs not emitted	→ Missing insights	Remedy: Tests, linters, standards
 Pipeline down / lag	→ Delayed visibility	Remedy: HA, backpressure, alerts
 Index missing fields	→ Unsearchable	Remedy: Schema tests, mapping
 Log explosion (noise)	→ High cost, slow	Remedy: Sampling, rate limits
 PII leaked	→ Compliance risk	Remedy: Redaction, scanners
 Clock skew	→ Wrong timelines	Remedy: NTP, use UTC
 Retention misconfig	→ Data loss / high cost	Remedy: Lifecycle, reviews
Access misconfig	→ Data exposure	Remedy: Least privilege, audit

STRUCTURED ✓ CONTEXTUAL ✓ CORRELATED ✓ QUERYABLE ✓ COST-AWARE ✓ COMPLIANT ✓ ACTIONABLE

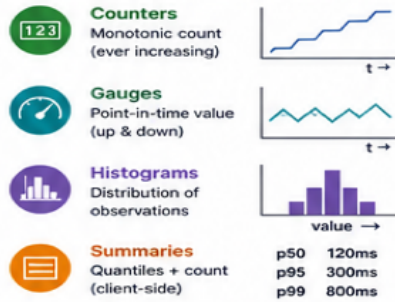
METRICS

1. WHY

- Make systems observable → Understand, decide, and improve.
- Visibility** See what's happening in real time
- Debugging** Find and fix issues faster
- Performance** Track health and user experience
- Capacity** Plan ahead, avoid surprises
- Reliability** Detect degradation, prevent outages

What you don't measure, you can't improve.

2. CORE IDEA



Dimensions & Cardinality

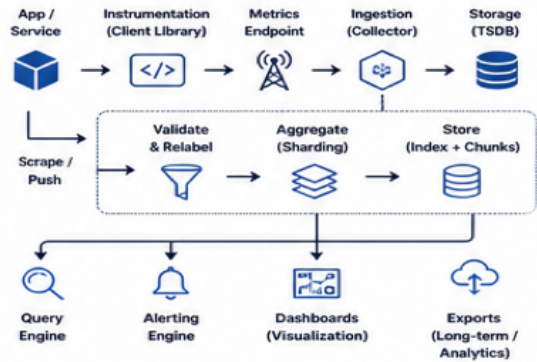
Dimensions = key-value pairs
(region=us, status=500)

Cardinality = # of unique time series
Keep it low!

Good metrics: actionable, low cardinality, correct, and cheap.

3. INTERNALS

End-to-end Metric Flow



Pull or push → Validate → (Optionally) aggregate → Store → Query → Alert → Visualize

4. COMPLEXITY

Cost Drivers

- Cardinality** (# of time series) **High ↑**
- Ingestion rate** (samples/sec) **High ↑**
- Retention** (duration) **High ↑**
- Query cost** (range × series) **High ↑**

Big-O (rule of thumb)

Ingestion: $O(S)$ S = samples/sec

Storage: $O(S \times R)$ R = retention

Query: $O(S_q \times T)$ S_q = series scanned, T = time range

High cardinality is the #1 cost & performance killer.

5. TRADE-OFFS

Metric Type	Pros	Cons	Best For
Counters	Simple, cheap, accurate totals	Needs rate() for trends	Requests, errors, bytes
Gauges	Current state, instant value	Harder to analyze trends	Queue depth, connections, CPU, temp
Histograms	Great percentiles, aggregatable	Many buckets = more cost	Latency distributions
Summaries	Accurate quantiles	Not aggregatable across labels	Client-side latency

Other Trade-offs

- Resolution ↑ → Cost ↑
- Accuracy ↑ → Cost ↑
- Cardinality ↑ → Cost ↑, Performance ↓
- Pre-aggregate ↑ → Flexibility ↓
- Real-time alerts ↓ false negatives → More noise ↑

Choose the simplest metric that answers the question.

6. DISCUSSION

Questions to Explore

- ✓ What problem are we solving?
- ✓ Who will use this metric and how?
- ✓ What actions will it drive?
- ✓ What is an acceptable delay?
- ✓ What is acceptable error / cost?
- ✓ What are the cardinality drivers?
- ✓ How will it be validated?

Failure Modes

- ✗ Missing instrumentation
- ✗ High cardinality explosion
- ✗ Wrong semantics (reset, units)
- ✗ Clock issues / time skew
- ✗ Sampling bias
- ✗ Retention too short
- ✗ Alert fatigue (too noisy)

How to Validate

- ✓ Unit tests for metrics
- ✓ Load test & compare
- ✓ Golden signals check
- ✓ Backfill / replay test
- ✓ Dashboards review
- ✓ Alert fire drills
- ✓ SLO burn rate tests

7. RELATED

Metric Families

- RED** Rate, Errors, Duration
- USE** Utilization, Saturation, Errors
- Golden Signals** Latency, Traffic, Errors, Saturation

Alternatives / Complements

- Logs** Events with context (high cardinality)
- Traces** Request flow (high cardinality)
- Profiling** CPU/Memory hotspots
- Events** State transitions
- Business KPIs** Outcomes that matter

Metrics = aggregates. Use others for details.

8. PRODUCTION

Best Practices

- ✓ Prefer counters + histograms
- ✓ Use stable, low-cardinality labels
- ✓ Define clear naming convention
- ✓ Document metric semantics
- ✓ Version dashboards
- ✓ Set retention per use
- ✓ Monitor the observability stack
- ✓ Review cost regularly

Examples

E-commerce API (RED)

```
http_requests_total(method, status)
http_request_duration_seconds_bucket(le,...)
http_request_duration_seconds_count
http_request_duration_seconds_sum
```

Database (USE)

```
db_connections(state)
db_cpu_usage_ratio
db_saturation_ratio
db_errors_total(type)
```

Cache (USE)

```
cache_hit_ratio
cache_evictions_total
cache_memory_usage_bytes
cache_errors_total
```

Start small. Iterate. Remove what isn't used.

11. MENTAL HOOK

"COUNT. SEE. DISTRIBUTE. SUMMARIZE."



10. SENIOR NOTES

- Instrument to answer questions, not to collect everything.
- Protect the future you: low cardinality, clear semantics.
- Set SLOs based on user impact, not internal convenience.
- Balance cost, accuracy, and actionability continuously.
- Own the whole lifecycle: design → build → operate → evolve.

Good observability is a force multiplier.

9. INTERVIEW THINKING

Decision Flow



Clarify → Design → Defend → Iterate

RIGHT METRICS → RIGHT INSIGHT → RIGHT ACTION → BETTER SYSTEMS

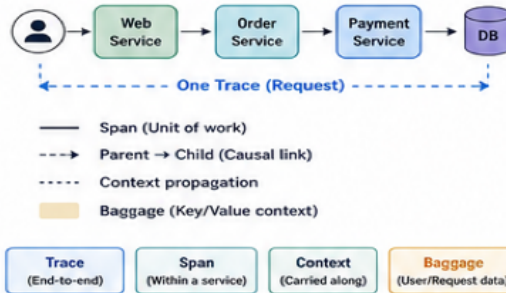
DISTRIBUTED TRACING

1 WHY

- 🎯 Observe requests across services & systems
- 🔍 Find latency, errors, & hops
- 📊 Debug in production
- 🛡️ Improve reliability & UX
- 💰 Prove performance, SLOs, & capacity

Tracing answers: *What happened, where, how long, and why slow?*

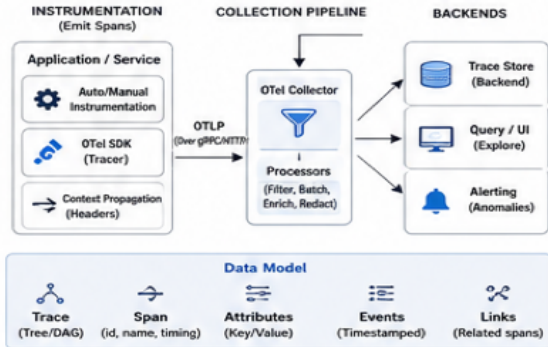
2 CORE IDEA



Trace = Tree/DAG of spans with timing, metadata & context

3 INTERNALS

High-level Architecture (OpenTelemetry)



4 COMPLEXITY

Time Complexity (per request)

- Instrumentation: ~ O(1) per span
- Context Propagation: ~ O(hops)
- Storage: ~ O(total spans)
- Query (Trace Lookup): ~ O(log N) (with index)

Space / Cost Drivers

- # of requests x # of spans
- Span attributes & events
- Retention period
- Sampling rate
- Cardinality of attributes

Cost grows with traffic, depth, and cardinality. Control with sampling, filtering, and limits.

5 TRADE-OFFS

Goal	Favor	Trade-off
🔍 Detail	Lower sampling rate (e.g., 100%)	Higher cost, more storage
💰 Cost	Higher sampling rate (e.g., 1%)	May miss rare issues
🕒 Performance	Head-based sampling	Misses slow outliers
📈 Catch tail latency	Tail-based sampling	Higher latency, more complexity
📁 Context richness	More baggage & attributes	Bigger payloads, PII risk
🛡️ Privacy	Minimize / redact baggage	Less debugging context



Start simple. Measure. Iterate.
Balance signal, cost, and overhead.

6 DISCUSSION

When to use Distributed Tracing

- ✓ Request crosses service/process/network boundary
- ✓ Need end-to-end latency breakdown
- ✓ Troubleshooting errors or intermittent slowness
- ✓ Correlate with logs, metrics, and alerts

Alternatives / Complements

- ➔ Logs: detailed events, point-in-time
- ➔ Metrics: aggregated trends & RED/USE
- ➔ Profiles: CPU / memory / lock contention
- ➔ Synthetic monitoring: external perspective
- ➔ **Correlation IDs only:** minimal visibility



Tracing answers "why slow?", others answer "what & where".

7 RELATED

- 🔗 Distributed Context Propagation (W3C Trace Context)
- 📁 Baggage (Key/Value context)
- 🌐 Service Mesh Observability (eBPF, proxies)
- 📊 Correlation with Logs & Metrics (Trace ID / Span ID)
- 📈 SLOs & Tail Latency (p95/p99/p999)
- 🔧 OpenTelemetry (OTel) Standard (SDK, API, Collector, OTLP)

8 PRODUCTION

Instrumentation

- Auto-instrument when possible
- Instrument at boundaries
- Name spans with high-cardinality caution
- Add meaningful attributes (not everything)

Sampling

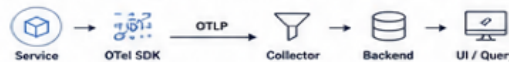
- Head-based for cost control
- Tail-based for latency/errors
- Always sample errors (100%)
- Dynamic sampling rules

Data Hygiene

- Redact PII in headers/baggage
- Drop noisy/low-value spans
- Set attribute & event limits
- Use semantic conventions

Operations

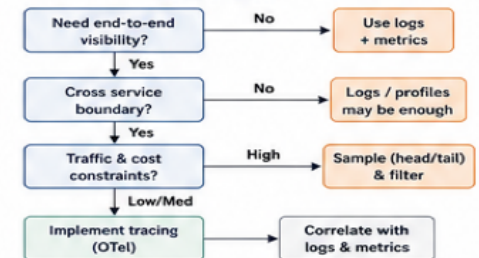
- Dashboards: latency & volume
- Alerts on error rate / p95 / p99
- Monitor ingestion & costs
- Retention aligned to needs



Measure impact: overhead < 5% target (typical)

9 INTERVIEW THINKING

Decision Flow



Show judgment on depth, cost, and risk.

10 SENIOR NOTES

- ★ Design for usability: meaningful names, hierarchies, and attributes.
- 📁 Control cardinality: user IDs, URLs, error messages — be deliberate.
- 📈 Use tail-based sampling for p99+/errors.
- 🔗 Propagate context across async (messaging, jobs) with care.
- 💎 Elevate signals, not data. Build golden traces for key journeys.
- 🔒 Govern with policies: retention, PII, access, and cost budgets.

11 MENTAL HOOK

A Trace is a **Story** of a Request.



Trace = Spans + Context + Time

- ✓ Follow the path
- ✓ Understand the context
- ✓ Find the slowdown

Key Terms

Trace	End-to-end collection of spans for a request
Span	Unit of work with timing & metadata
Trace ID	Unique ID for the entire trace
Span ID	Unique ID for a span
Parent / Child	Causal relationship between spans
Context Propagation	Carrying Trace ID, Span ID, baggage
Baggage	Key/Value data across services
Sampling	Deciding which traces/spans to record
Tail Latency	High-percentile latency (p95/p99/p999)
OTel / OTLP	OpenTelemetry / Transport Protocol



Observe
Don't guess.



Measure
Continuously.



Filter
Focus on signal.



Protect
Data & privacy.



Optimize
Iterate & improve.



Outcomes
Reliability & user value.